

No Magic!

To expose complexity helps users and developers.

A practical Concurrent Queue and How It Can (Or Can't) Be Implemented Using P0260R19.

KLUGC++, April 27th, 2026

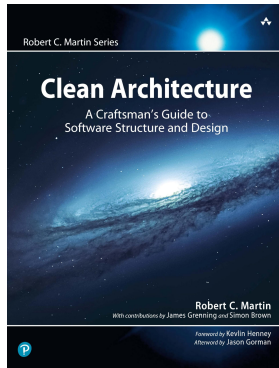
`mirko.rahn@itwm.fraunhofer.de`

`github.com/mrahn/klugc-25`

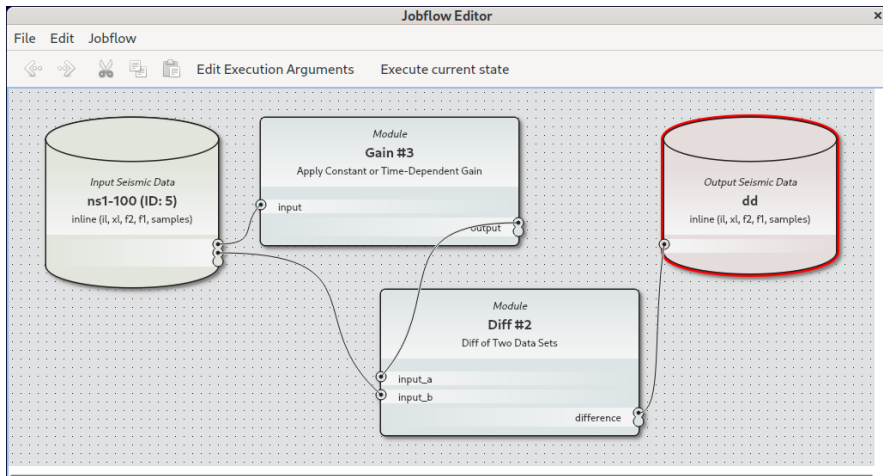
What is Magic?

To hide complexity means to take decisions (too) early.

- store configuration in home folder
- use detected hardware, use all available cores
- global parameters, parameter heuristics
- ✓ dozends of MPI configuration parameters
- ✓ fftw calibration run
- ✓ require complete configuration (file)

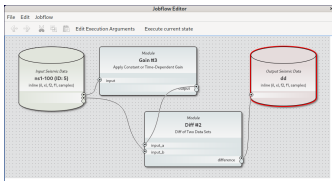


Report Progress: Applications are hierarchical and are composed of multiple tasks

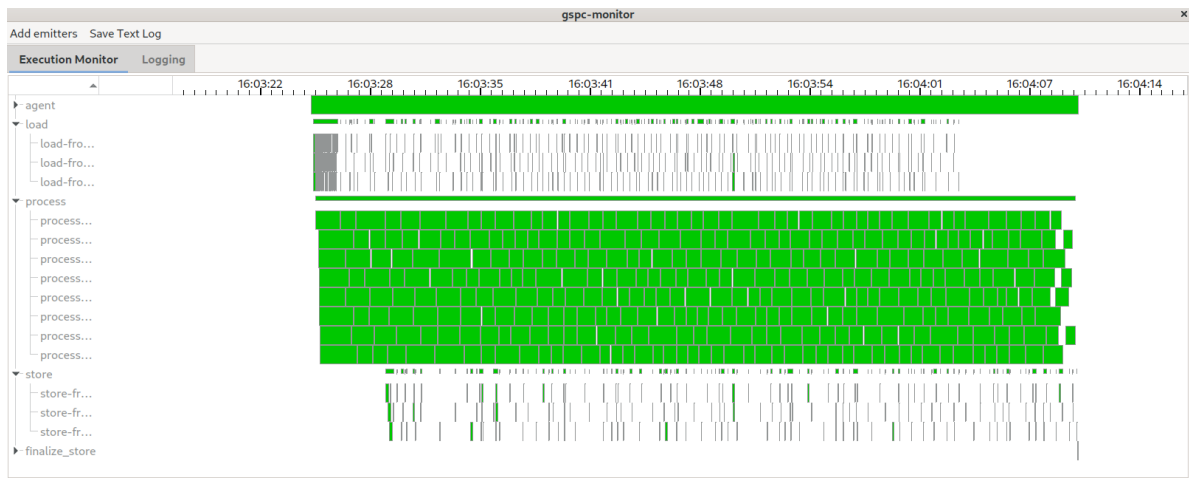


Executing jobflow

```
Progresses [.....] DONE STEPS: 0 of 1 (0%)
{ * Partition: [Gain #3, Diff #2]
  = [=====>] SUM: 10531290 of 11371501 (92.61%)
    - [=====>.] Process [Gain #3]: 2071426 of 2274300 (91.08%)
    - [=====] Load [Gain #3/input <- 'ns1-100']: 2274300 of 2274300 (100%)
    - [=====>.] Store [Diff #2/difference -> 'dd']: 1955632 of 2274300 (85.99%)
    - [.....] Finalize [Diff #2/difference -> 'dd']: 0 of 1 (0%)
    - [=====>.] Process [Diff #2]: 1955632 of 2274300 (85.99%)
    - [=====] Load [Diff #2/input_b <- 'ns1-100']: 2274300 of 2274300 (100%)
}
```



Report Status : Multiple activities happen at the same time



TaskQueue: Textbook interface, single thread of execution

```
class TaskQueue
{
    auto empty() const -> bool;
    auto push (Task) -> void;
    // pre: !empty()
    auto pop() -> Task;
}

tasks.push (root);
single_prosumer:
    while (!tasks.empty())
    {
        auto const task {tasks.pop()};

        for (auto child : execute (task))
        {
            tasks.push (child);
        }
    }
    // FINE
```

TaskQueue: Textbook interface, single thread of execution

```
class TaskQueue
{
    auto empty() const -> bool;
    auto push (Task) -> void;
    // pre: !empty()
    auto pop() -> Task;
}

tasks.push (root);
single_prosumer:
    while (!tasks.empty())
    {
        handle (task.pop());
    }
    // FINE
```

ConcurrentTaskQueue: Textbook interface, multiple threads of execution

```
class ConcurrentTaskQueue
{
    auto empty() const -> bool;
    auto push (Task) -> void;
    // pre: !empty()
    auto pop() -> Task;
}

tasks.push (root);
multiple_prosumers_at_the_same_time:
while (!tasks.empty())
{
    handle (tasks.pop()); // PRECONDITION VIOLATION!
}
```


ConcurrentTaskQueue: Textbook interface, k many coordinated threads of execution

```
class ConcurrentTaskQueue
{
    // BUMMER! empty() must not exist
    auto empty() const -> bool;
    auto push (Task) -> void;
    // pre: !empty()
    auto pop() -> Task;
}

auto done {Atomic {0}};
tasks.push (root);
k_many_prosumers_at_the_same_time:
while (done < k)
{
    if (tasks.empty())
    {
        done += 1;
        while (done < k && tasks.empty()) { yield(); }
    }
    if (!tasks.empty())
    {
        done -= 1;
        handle (tasks.pop()); // PRECONDITION VIOLATION!
    }
}
```

ConcurrentTaskQueue: pop might return nothing, multiple threads of execution

```
class ConcurrentTaskQueue
{
    auto push (Task) -> void;
    // Nothing if empty
    auto pop() -> optional<Task>;
}

tasks.push (root);
multiple_prosumers_at_the_same_time:
while (auto const task {tasks.pop()})
{
    handle (*task);
}
// BUMMER! ONE THREAD DOES ALL THE WORK!
```

ConcurrentTaskQueue: Counting client, multiple threads of execution

```
class ConcurrentTaskQueue
{
    struct Client
    {
        auto push (Task) const -> void;
        // Nothing if empty forever
        auto pop() const -> optional<Task>;
    };
    auto client() -> unique<Client>;
}
```

```
tasks.client()->push (root);
multiple_prosumers_at_the_same_time:
    auto const client {tasks.client()};
    while (auto const task {client->pop()})
    {
        handle (*task);
    }
    // FINE
```

P0260R19: Relevant API for this talk

@AI: Please add a slide with a brief summary about the relevant API of P0260.

```
std::bounded_queue<Task> q {N};

q.push (task);           // bool
q.pop();                 // optional<Task>

q.close();               // end-of-stream
q.is_closed();           // observer

q.try_push (task);       // conqueue_status
q.try_pop();             // expected<Task, status>

q.async_push (task);
q.async_pop();
```

- No `empty()`: that would be racy.
- `pop()` blocks until data or `close()`.
- After `close()`, queued tasks can still be popped.
- Once empty and closed, `pop()` returns `nullopt`.
- No standard `Client`/front/back type in R19.
- Client counting would have to decide when to call `close()`.

Quiescence is not Close

@AI: Please add a slide “Quiescence is not Close” and describe the differences.

TaskQueue quiescence

- `pop()` returns nothing if no client can make progress now.
- This is detected from global state of clients and waiting `pop()` calls.
- There is no permanent closed state in the interface.
- A later `push()` by an old or new client can start a second round.

P0260 close

- `close()` is explicit and one-way.
- After `close()`, every later `push()` fails.
- `pop()` returns nothing only when the queue is empty and closed.
- No second round on the same queue object is possible.

ConcurrentTaskQueue: Counting client, multiple threads of execution

```
class ConcurrentTaskQueue
{
    struct Client
    {
        auto push (Task) const -> void;
        //                +-----+
        // Nothing if empty | FOR EVER |
        //                +-----+
        auto pop() const -> optional<Task>;
    };
    auto client() -> unique<Client>;
}
```

```
tasks.client()->push (root);
multiple_prosumers_at_the_same_time:
    auto const client {tasks.client()};
    while (auto const task {client->pop()})
    {
        handle (*task);
    }
    // FINE

tasks.client()->push (root2);
// ^BUMMER: Second round interleaves
```

TaskQueue vs Adapter

@AI: Please add a slide that summarizes the differences between the TaskQueue and the Adapter.

Current TaskQueue

- Owns `deque`, mutex, condition variable, and close state.
- Decides quiescence directly inside `pop()`.
- Wakes blocked clients directly when closing.
- Queue mechanics and client counting are tightly coupled.

P0260-based Adapter

- Underlying queue owns blocking `pop()`, `close()`, and wakeup.
- Adapter keeps only client-aware counters and quiescence logic.
- Quiescence is translated into one call to `q.close()`.
- Most queue machinery moves from the wrapper into the P0260 queue.

P0260 carries most of the queue machinery, the adapter adds only the client-based termination rule.

No Magic!

- Requires that users know what they do.
- Requires that developers know what they do.
- Helps users to know what they do.
- Helps developers to know what they do.
- Leads to better users experience , short term and long term.
- Leads to simpler codes , short term and long term.
- AI: P0260 carries most queue mechanics, while client-based termination stays explicit in the adapter.